1. الـ Embedding (Denormalization)

هنا إنت بتحط الـ Data المتعلقة ببعضها جوه وثيقة واحدة (Single Document).

- **القاعدة:** "Data that is accessed together should be stored together."
- **الشكل:** الـ Document الأب بيحتوي على Array أو Sub-document فيه كل التفاصيل.

امتى تدوس فيها؟

- **One-to-Few:** لما يكون عندك علاقة بسيطة (مثلاً مستخدم وعناوين الشحن بتاعته).
- **Read Performance:** لما تكون عايز تجيب كل الداتا بـ Query واحدة من غير `lookup` (الـ Join اللي هي الـ في مونجو).
- **Atomicity:** التعديل على الـ Document الواحد بيكون "Atomic"، يعني يا يتنفذ كله يا ميتنفذش خالص.

الخطر (The Trap):

- **Document Size Limit: 16 MB** المونجو بيلزمك بـ Document. كحد أقصى للـ Array لو الـ دي بتكبر للملانهاية (مثلاً بوست والكومنتات بتاعته لو هو مشهور)، الـ Document هيضرب منك.
- **Data Duplication:** لو غيرت معلومة في حتة، هتضطر تلف تعدلها في كل الـ Documents اللي عاملة لها Embed.

---

2. الـ Referencing (Normalization)

هنا بتفصل الداتا في Collections مختلفة، وبتحط ID بس (غالباً `_id`) كـ "لينك" بينهم.

- **القاعدة:** "Divide and Conquer."
- **الشكل:** الـ Document بيحتوي على `ObjectId` بيشاور على مكان الداتا في Collection تانية.

امتى تدوس فيها؟

- **One-to-Many / Many-to-Many:** لما يكون الـ "Many" (مثلاً متابعين بلوجر مشهور) ده عدده ضخم جداً.
- **Data Consistency:** لما تكون الداتا بتتغير كتير ومحتاج تعدلها في مكان واحد بس (Single source of truth).
- **Large Data Sets:** عشان تتجنب حد الـ 16 MB وتخلي الـ Documents خفيفة.

الخطر (The Trap):

- **Performance Hit:** دي عملية مكلفة على الـ `$lookup`. وتستخدم `aggregate` عشان تجمع الداتا هتحتاج تعمل Pipeline عشان توصل للي إنت عايزه لو مش مستخدم الـ RAM والـ CPU لو الداتا ضخمة.
- **Multiple Queries:** أحياناً هتضطر تعمل أكتر من Query عشان توصل للي إنت عايزه لو مش مستخدم الـ صح.

1. `db.collection.insertOne()`

بتستخدمها لما يكون معاك **Document واحد** بس عايز توثقه.

- **الـ `_id` :** لو نسيت تحطه، المونجو بيعملك جنريت لـ `ObjectId` فريد تلقائي. ده بيضمن إن كل سجل ليه بصمة متميزة.
- **مثال سريع:**

JavaScript

```
db.products.insertOne({
  item: "laptop",
  qty: 15,
  tags: ["electronics"]
})
```

---

2. `db.collection.insertMany()`

دي بتاعة المهام التقيلة لما يكون معاك **Array of Documents**.

- **السرعة:** أوفر بكتير من إنك تبعت طلب لكل Document لوحده، بتقلل الـ Overhead على السيرفر والشبكة.
- **الـ Ordered Inserts:** باي ديفولت، المونجو بيمشي بالترتيب. لو رقم 3 فيه غلط، العملية بتقف ورقم 4 مش بيدخل. لو عايز "كلو يدخل واللي يضرب يضرب"، بتستخدم `{ ordered: false }` .
- **مثال سريع:**

JavaScript

```
db.products.insertMany([
  { item: "mouse", qty: 50 },
  { item: "keyboard", qty: 20 },
  { item: "monitor", qty: 10 }
])
```

الخلاصة الفنية:

- **الـ Schema-less:** مش لازم تعرف العواميد (Fields) قبل ما تبدأ، كل Document ممكن يكون ليه شكل مختلف، بس الأفضل توحد عشان الـ Querying بعدين.
- **الـ Atomicity:** العمليات دي Atomic على مستوى الـ Document الواحد، يعني يا يدخل كله يا ميتحطش خالص.

```
static fetchAll() {
  const db = getDb();
  return db.collection("products")
    .find() // بيرجع Cursor
    .toArray() // بيحول الـ Cursor لـ Array (مناسب للداتا الصغيرة والمتوسطة)
    .then(products => products)
    .catch(err => console.log("FetchAll Error:", err));
}

static findById(prodId) {
  const db = getDb();
  return db.collection("products")
    .find({ _id: new mongodb.ObjectId(prodId) }) // لازم ObjectId
    .next() // هات أول وثيقة واقفل Cursor
    .then(product => product)
    .catch(err => console.log("FindById Error:", err));
}
```

1. الـ `find()` والـ Cursor

لما بتنادي `.find()`، المونجو مش بيروح يجيب كل الداتا ويرميها في الـ Memory مرة واحدة. هو بيرجع حاجة اسمها **Cursor**.

- الـ Cursor ده عامل زي "مؤشر" بيسمحلك تتحرك جوه الداتا.
- لما بتستخدم `.toArray()`، إنت بتقول للمونجو: "حول كل اللي لقيتهم لـ Array في الميموري عندي".
- **تحذير فني:** لو عندك مليون منتج، `.toArray()` هتنفخ الـ RAM عندك. في الحالات دي بنستخدم Pagination (يعني `limit` و `skip`).

2. الـ `findById` وحوار الـ `ObjectId`

إنت عملت الصح هنا إنك استخدمت `new mongodb.ObjectId(prodId)`.

- **ليه؟** لأن المونجو بيخزن الـ `_id` كـ BSON Binary، مش String عادي. لو بعت الـ ID كـ String، المونجو هيقولك "مش لاقي حاجة".
- **الـ `.next()`:** بما إن الـ `find()` بترجع Cursor، والـ Cursor ده ممكن يكون فيه كذا نتيجة، فإنت بتقول له "هات لي أول واحد يقابلك" وده أداء (Performance) عالي جداً لأنك مش محتاج تحول لـ Array عشان تجيب عنصر واحد.

```
save() {
    // Obtain the active database connection instance
    const db = getDb();
    let dbOp;
```

```
    if (this._id) {
      // UPDATE PATH: If _id exists, we are modifying an existing document
      // We use updateOne with two arguments: the filter and the update
operation
      dbOp = db
        .collection("products")
        .updateOne(
          { _id: new mongodb.ObjectId(this._id) }, // Filter: Convert string
ID to BSON ObjectId
          { $set: this } // Atomic operator: Updates only the fields provided
in 'this'
        );
    } else {
      // INSERT PATH: No _id means this is a new product
      // MongoDB will automatically generate a unique _id for the document
      dbOp = db.collection("products").insertOne(this);
    }

    // Execute the operation and return the promise for the controller to
handle
    return dbOp
      .then((result) => {
        // Log operation metadata (e.g., matchedCount, modifiedCount, or
insertedId)
        console.log(result);
      })
      .catch((err) => {
        // Catch network or database-level errors
        console.log(err);
      });
  }
```

- **الـ Logic:** إنت بتعتمد على وجود `_id` . لو موجود، يبقى المنتج ده "قديم" وعايز يتعدل (`updateOne`). لو مش موجود، يبقى "جديد" لنج وهينزل داتابيز لأول مرة (`insertOne`).
- **الـ `$set` Operator :** دي أهم حتة في الـ Update. لو بعت `this` من غير `$set` في `updateOne` المونجو هيمسح الـ Document كله ويحط مكانه الأوبجكت الجديد، لكن `$set` بتعدل الـ Fields اللي اتغيرت بس وبتحافظ على الباقي.
- **تركة الـ `_id` في التعديل:** خلي بالك وأنت بتبعت `this` جوه الـ `$set` . المونجو درايفر ساعات بيضايق لو لقيت `_id` جوه الأوبجكت اللي بيتعمل له Update (لأن الـ `_id` Immutable مبيتغيرش). لو ضربت معاك، ابقى ابعت الـ fields فرادي أو امسح الـ `_id` من الأوبجكت قبل الـ update.

```
app.use((req, res, next) => {
  User.findById("69da9d9156d3c8f65fa1b93f")
    .then((user) => {
      // 1. Store the user object in the request to use it everywhere
      req.user = user;
      // 2. ONLY call next() here, after the user is successfully found
      next();
    })
    .catch((err) => {
      console.log(err);
      // If DB fails, we still need to decide if we should stop or continue
      // next(); // You could call next() here too if you want to proceed
without a user
    });

  // REMOVE the next() that was here!
});
```

=> more on source code

عشان نفهم الـ **Create Operations** من منظور الـ Source Code والـ Architecture، لازم نفرق بين الـ (`insertOne` , `insertMany`) High-level API وبين الـ Plumbing اللي بيحصل تحت في الموتور.

لما بتنادي `db.collection.insertOne()` ، المونجو مش بس بيحدف داتا في فايل، ده بيعدي بمراحل معقدة عشان يضمن الـ **Durability** والـ **Performance**.

1. الـ High-Level: الـ API Layer

في الـ Repo، العمليات دي بتبدأ في `src/mongo/db/commands/insert_cmds.cpp` .

- **Validation:** ومش واصل للحد الأقصى "Valid BSON" ده Document أول حاجة المونجو بيعملها هي التأكد إن الـ (16MB).
- **Auto-Collection Creation:** لو الـ Collection مش موجودة، الـ `Database` Object بيستدعي `createCollection()` داخلياً قبل ما يكمل عملية الإدخال.
- **_id Generation:** لو إنت مبعتش `_id` في الـ Document، الـ Driver أو الـ Server (حسب الـ Implementation) بيولد `ObjectId` فريد.

---

2. الـ Mid-Level: الـ Execution Engine

هنا الكود بينزل لـ `src/mongo/db/ops/insert.cpp` . المونجو بيبدأ يجهز الـ **Write Conflict Framework**:

- **Concurrency Control:** المونجو بيستخدم نظام "Optimistic Concurrency Control". هو بيفترض إن مفيش حد هيعدل في نفس الـ Document دلوقتي، فبيبدأ يجهز الـ Write.
- **The Plan:** الـ Insert مش محتاجة Query Optimizer كبير، لكنها محتاجة تحديث للـ **Indexes**. لو الـ Collection بالإضافة للداتا الأساسية Indexes بتوع الـ B-Trees المونجو لازم يروح يكتب في الـ 5، 5 Indexes عليها.

---

3. الـ Low-Level: الـ "The Muscle" WiredTiger"

ده أعمق جزء في الكود `src/mongo/db/storage/wiredtiger` . العملية بتمشي كالتالي:

1. **Memory First (The Cache):** الـ Document الجديد بيتكتب الأول في الـ **WiredTiger Cache**. الداتا بتكون في الميموري عشان السرعة الخرافية.
2. **The Journal (WAL):** المونجو بيكتب الـ ،(اللي ذكرتها AWS زي مشكلة) عشان نضمن إن الداتا مطرش لو النور قطع سريعة جداً (Sequential Write) على الهارد ديسك. ده كتابة تسلسلية **Journal File** فوراً في الـ Operation.
3. **B-Tree Insertion:** الـ WiredTiger بيحدث الـ B-Tree واللي شايلة الداتا والـ B-Trees بتاعة الـ Indexes.
4. **Checkpointing:** "Checkpoint" بيعمل الـ WiredTiger، (يوصل لحجم معين Journal أو لما الـ) كل 60 ثانية الأساسية `.wt` بياخد فيه كل اللي في الميموري ويرصه بشكل نهائي ومنظم في ملفات الـ.

الفرق بين `insertOne` و `insertMany` في الكود:

- **insertOne:** هي عملية Unitary. بتبعت Document واحد، بيخلص الـ Pipeline بتاعه وبيرجع `Acknowledgement` .
- **insertMany:** المونجو بيعمل لها **Bulk Write**. بدل ما يفتح Transaction لكل Document، بيحاول يجمعهم مع بعض (Batching) عشان يقلل الـ Network Overheads والـ Lock Contention في الـ Storage Engine. ده بيخلي الـ `insertMany` أسرع بكتير من إنك تعمل Loop وتنادي `insertOne` .